

Plan de Pruebas y Resultados

Sistema de Gestión Hotelera — Paradise Resort

Versión 1.0 · julio de 2026

Universidad Latina de Costa Rica Análisis y Diseño de Sistemas II

1. Alcance

Este documento recoge la estrategia de pruebas definida en la Etapa 2 y los resultados obtenidos sobre la versión 1.0 del sistema. Se ejecutaron tres niveles:

Nivel	Cantidad	Ejecución
Unitarias	102	Automatizada (xUnit + Moq)
Integración	109	Automatizada, contra base de datos real
Aceptación (UAT)	27	Sobre el sistema en funcionamiento, siguiendo los flujos de los wireframes

Los conteos incluyen cada fila de los casos parametrizados ([Theory]), que es como los ejecuta el corredor de pruebas.

2. Criterios de aceptación

Conforme a la Etapa 2, una versión está lista para desplegarse cuando:

- Supera el **100 %** de las pruebas críticas de integración.
- Alcanza una cobertura mínima del **80 %** del código fuente.
- No presenta fallos graves de seguridad en el análisis estático.
- Cumple los tiempos de respuesta establecidos en RNF02.

Los cuatro criterios se cumplen.

3. Pruebas automatizadas

3.1 Resultado

Passed! - Failed: 0, Passed: 211, Skipped: 0, Total: 211

Estables en cinco ejecuciones consecutivas.

3.2 Cobertura

87,0 % del código propio (meta: 80 %).

Capa	Cobertura
Domain	95,3 %
API	94,2 %
Shared	87,8 %
Persistence	86,2 %
Infrastructure	85,4 %
Application	84,0 %

La medición se reproduce con:

```
dotnet test backend/HotelParadiseResort.sln --settings backend/cobertura.runsettings --collect:"XPlat
```

El archivo de configuración excluye las migraciones de Entity Framework, que son código generado por la herramienta en tiempo de diseño: solo el snapshot del modelo son unas 1830 líneas de declaraciones que ningún caso de prueba ejecuta. Contarlas rebajaría la cifra al 79 % sin que eso midiera nada real, pues el esquema que producen queda verificado por las 109 pruebas de integración, que crean la base de datos a partir de esas mismas migraciones. No se excluye ninguna otra cosa; en particular, se contabilizan íntegros los cuerpos de los métodos asíncronos.

3.3 Distribución

Conjunto	Casos	Qué verifica
Dominio/	65	Los seis patrones GoF: transiciones de estado, cálculo de tarifas, cadena de decoradores, construcción de facturas, plantilla de reportes
Aplicacion/	16	Reglas de reservas y de administración de usuarios, con dobles de prueba
Seguridad/	11	Derivación PBKDF2, sal aleatoria, rechazo de hashes mal formados
Api/	10	Traducción de cada excepción a su código HTTP; no exposición de trazas en producción
Integracion/	109	API ↔ base de datos de punta a punta

3.4 Entorno de las pruebas de integración

Se ejecutan contra una base de datos **real y dedicada** (`HotelDB_Pruebas`), que la propia suite crea y elimina. No se emplea una base en memoria porque ocultaría justamente lo que estas pruebas deben verificar: índices únicos, restricciones CHECK, claves foráneas, concurrencia optimista y collation.

```
PRUEBAS_SQL_PASSWORD=<contraseña-del-motor> dotnet test backend/HotelParadiseResort.sln
```

3.5 Tiempos de respuesta (RNF02)

Medidos sobre la API en ejecución, contra la base de datos real. El límite establecido es de **2,5 segundos**.

Operación	Tiempo	Código
Panel principal (indicadores)	0,007 s	200
Consulta de disponibilidad, 90 días	0,003 s	200
Listado de clientes paginado	0,005 s	200
Listado de reservas paginado	0,004 s	200
Reporte de ocupación, 90 días	0,004 s	200
Reporte de ingresos, 90 días	0,003 s	200
Reporte de temporadas, 90 días	0,003 s	200
Reporte de ocupación, 3 años (rango máximo permitido)	0,004 s	200
Catálogo de habitaciones	0,003 s	200

Todas las operaciones se sitúan tres órdenes de magnitud por debajo del límite. Conviene precisar el alcance de esta medición: se tomó con el volumen de datos de demostración (20 habitaciones y unas decenas de reservas) y con un solo cliente, de modo que acredita que no hay consultas N+1 ni recorridos de tabla completos, pero **no** constituye una prueba de carga. La verificación de los 50 usuarios concurrentes de RNF03 se sostiene sobre el diseño —bloqueo optimista por fila con `rowversion` y transacciones serializables en la comprobación de disponibilidad—, cuyo comportamiento ante escritura simultánea sí está cubierto por las pruebas de integración. Una prueba de carga sostenida sobre volúmenes de producción queda propuesta para la puesta en marcha.

4. Pruebas de aceptación (UAT)

Recorren los flujos operativos que el personal ejecuta a diario, siguiendo los wireframes de la Etapa 2. Se ejecutaron contra el sistema desplegado y la base de datos real.

4.1 Resultados

ID	Caso de prueba	Resultado esperado	Obtenido	Estado
CP-01	Inicio de sesión con credenciales válidas	200	200	✓
CP-02	Inicio de sesión con contraseña incorrecta	401	401	✓
CP-03	El panel principal devuelve sus indicadores	200	200	✓
CP-04	Registrar cliente con acentos y ñ	201	201	✓
CP-05	Buscar sin acentos encuentra al cliente	Lo encuentra	Lo encuentra	✓
CP-06	La consulta de disponibilidad devuelve habitaciones	Lista no vacía	Lista no vacía	✓
CP-07	La tarifa se calcula como noches × precio	Coincide	Coincide	✓
CP-08	Reservar para el día en curso	201	201	✓
CP-09	La reserva confirmada queda en estado «Confirmada»	Confirmada	Confirmada	✓
CP-10	Sobre-reserva rechazada (regla central del sistema)	409	409	✓
CP-11	Check-in sobre reserva confirmada	201	201	✓
CP-12	Tras el check-in la habitación pasa a «Ocupada»	Ocupada	Ocupada	✓
CP-13	Registrar un consumo en la estadía	200	200	✓
CP-14	El monto del consumo es cantidad × precio unitario	\$31.00	\$31.00	✓
CP-15	La cuenta suma hospedaje más consumos	Coincide	Coincide	✓
CP-16	Facturar sin check-out previo es rechazado	422	422	✓
CP-17	Registrar el check-out	200	200	✓
CP-18	Emitir factura con descuento justificado	201	201	✓
CP-19	El total resta correctamente el descuento	Coincide	Coincide	✓
CP-20	La factura con pago inmediato queda «Pagado»	Pagado	Pagado	✓
CP-21	Doble facturación de una estadía es rechazada	409	409	✓
CP-22	El recepcionista no accede a los reportes	403	403	✓
CP-23	El recepcionista sí accede a clientes	200	200	✓
CP-24	Reporte de ocupación con indicadores y series	Completo	Completo	✓
CP-25	Reporte de ingresos con indicadores y series	Completo	Completo	✓

ID	Caso de prueba	Resultado esperado	Obtenido	Estado
CP-26	Reporte de temporadas con indicadores y series	Completo	Completo	✓
CP-27	Endpoint protegido sin token devuelve 401	401	401	✓

Resultado: 27 de 27 casos superados.

4.2 Verificación de persistencia

Se creó un cliente («Begoña Muñoz Céspedes») **desde el formulario de la interfaz** y se consultó después directamente la base de datos con SQL:

- El registro existe en la tabla `Cliente` con los acentos y la ñ correctos.
- `FechaCreacion` fue asignada por la auditoría automática del `DbContext`.
- Se generó sola la entrada correspondiente en `HistorialCliente`, con el usuario que la ejecutó.
- La búsqueda responde igual ante «Munoz», «Muñoz», «MUNOZ», «Begona», «Begoña» y «cespedes».

5. Defectos detectados y corregidos

Clasificados por severidad conforme a la Etapa 2.

#	Severidad	Defecto	Corrección	Estado
1	Bloqueante	El hospedaje se calculaba con los sellos de tiempo reales de entrada y salida en lugar de las noches contratadas. Una cuenta llegó a mostrar 27 noches	<code>Estadia.CalcularNoches()</code> y <code>ConstruirCuenta()</code> usan el período de la reserva	Cerrado
2	Mayor	El check-in no validaba que su fecha cayera dentro del rango reservado: se podía registrar la llegada de una reserva de meses después	Validación explícita del rango en <code>ServicioEstadias</code>	Cerrado
3	Mayor	El estado instantáneo de la habitación bloqueaba reservas futuras: una habitación «En limpieza» hoy no aparecía disponible para septiembre	Solo <code>EnMantenimiento</code> retira del inventario reservable; el estado real se revalida en el check-in	Cerrado
4	Mayor	Zona horaria: el sistema rechazaba reservas para el día en curso. La comparación se hacía contra UTC y, en Costa Rica (UTC-6), después de las 18:00 el día UTC ya había avanzado	Se añadió <code>FechaOperativa</code> al proveedor de fecha, expresada en la zona horaria del hotel, y las reglas de calendario la usan	Cerrado
5	Mayor	AutoMapper 12, 13 y 14 arrastraban CVE-2026-32933 (denegación de servicio, CVSS 7.5)	Se fijó la versión 15.1.1, parcheada	Cerrado
6	Menor	Intentar facturar dos veces devolvía 422 en una ruta y 409 en otra, para el mismo escenario	Unificado: estadía ya facturada → 409; estadía aún abierta → 422	Cerrado
7	Menor	El gráfico de reportes solo emitía las semanas con datos, de modo que aparecía «Sem 5» aislada sin las anteriores	La serie cubre todo el rango, con cero donde no hay actividad, y se rotula con la fecha de inicio de cada semana	Cerrado
8	Menor	Los importes se mostraban como «USD 318,75» en lugar de «\$318.75»	Formato del dólar con símbolo antepuesto	Cerrado
9	Menor	La interfaz mostraba valores en notación de código: «RedesSociales», «TarjetaCredito»	Función <code>etiquetaLegible</code> que traduce a texto natural	Cerrado
10	Menor	Los botones deshabilitados quedaban ilegibles: el degradado persistía y el texto atenuado no contrastaba	Se anula el degradado en estado deshabilitado	Cerrado
11	Trivial	Dos pruebas dependían del estado global y fallaban de forma intermitente según el orden de ejecución	Rediseñadas para crear sus propios datos; la regla del último administrador se movió a una prueba unitaria determinista	Cerrado

No quedan defectos abiertos.

6. Conclusión

El sistema supera los criterios de aceptación definidos en la Etapa 2:

- 211 pruebas automatizadas en verde, con 87,0 % de cobertura.
- 27 de 27 casos de aceptación superados sobre los flujos reales de operación.
- Sin advertencias de compilación ni vulnerabilidades conocidas en las dependencias.
- Once defectos detectados durante el desarrollo, todos corregidos y verificados.

La regla de negocio central —impedir dos reservas sobre la misma habitación en fechas solapadas— está verificada tanto en pruebas automatizadas como en aceptación, y se apoya en tres mecanismos independientes: transacción serializable, bloqueo optimista por fila e índice de detección de solapamientos.

Plan de Pruebas · Paradise Resort v1.0 · Universidad Latina de Costa Rica